Mobile Phone Retail and Sales Management Database

SQL PROJECT REPORT | SQLITE DATABASE | SYNTHETIC RETAIL DATA

Student name	Kommineni Sathvika	**Student ID**	25021550
Git repository link	https://github.com/sathvika11484/Data-Mining-and-Discovery		

1. Project aim and context

This project models the information needed by a fictional mobile-phone retailer. Staff need to maintain a product catalogue, customer records, store locations, sales orders and stock at each location. Keeping all of these details in one table would repeat customer and product information for every purchase, making updates harder and increasing the risk of inconsistent records. The relational design separates these business areas and connects them through keys.

The database can answer practical management questions: which brands generate the most completed-sale revenue, which models sell most often, how monthly sales change, which payment methods are used and which store-product combinations may need replenishment. All records are generated for this assignment; they are not actual customer records or evidence of UK retail trends.

2. Database structure and design choices

customers stores each fictional customer once, including name, email, age band, region, customer-since year and marketing preference. **products** describes each phone variant because storage, colour and condition can affect product identity and price. **stores** records retail branches and an online fulfilment location. **orders** stores the transaction header: customer, store, date, channel, payment method, delivery estimate and status.

The products within an order are stored in **order_items**, allowing one order to contain several products without adding repeated product columns to the order table. The line stores the price paid at the time of sale, preserving sales history if the catalogue price later changes. **store_inventory** records the quantity of each product held at each store, together with a reorder level and last-restocked date. Separating these entities reduces duplication and update anomalies.

3. Entity relationship diagram

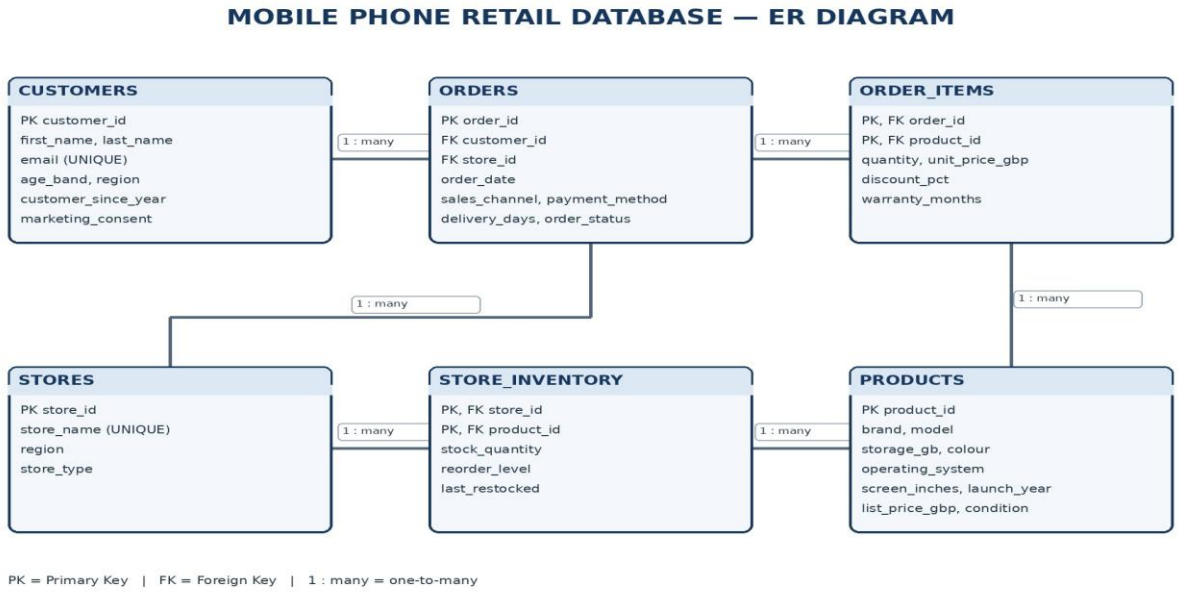


Figure1:Six related tables; order_items and store_inventory resolve many-to-many relationships.

4. Keys, relationships and validation

customers, products, stores and orders each have a primary key. Two associative tables use composite primary keys: (order_id, product_id) in order_items and (store_id, product_id) in store_inventory. Both columns are required to identify a unique row. products also has a composite UNIQUE constraint across brand, model, storage_gb, colour and product_condition, preventing the same phone variant from being entered twice.

Foreign keys connect orders to customers and stores, order_items to orders and products, and store_inventory to stores and products. NOT NULL rules protect required fields; UNIQUE rules protect customer email addresses and store names; CHECK rules restrict categories and sensible numeric ranges. For example, quantity must be between 1 and 3, prices must be positive, and marketing_consent must be 0 or 1. Referential integrity is checked with PRAGMA foreign_key_check.

5. Table overview

Table	Main information	Key arrangement
customers	Name, email, age band, region, marketing choice	PK customer_id; unique email
products	Brand, model, storage, colour, OS, screen, year, price, condition	PK product_id; composite UNIQUE
stores	Store name, region and type	PK store_id; unique store name
orders	Customer, store, date, channel, payment, delivery estimate, status	PK order_id; customer/store FKs
order_items	Products per order, quantity, sale price, discount, warranty	Composite PK (order_id, product_id)
store_inventory	Stock by store/product, reorder level, restock date	Composite PK (store_id, product_id)

6. Dataset and data types

The database contains 320 customers, 63 products, 9 stores, 1,535 orders, 1,748 order_items and 567 store_inventory records. order_items exceeds the 1,000-row requirement. Values vary by brand, model, region, price, date, channel, payment method, discount and stock level. Prices are generated examples, not current market quotations.

Measurement scales: nominal—brand, colour, operating system, region, store type, channel and payment method; ordinal—age band, product condition and order status; interval—launch year and customer-since year; ratio—price, screen size, storage, quantity, discount, delivery days and stock quantity.

SQLite storage types: TEXT for names, categories and ISO dates; INTEGER for identifiers, years, quantities and consent flags; REAL for prices, screen size and discounts.

7. Deliberate data-quality cases

The generator deliberately creates 59 missing delivery estimates in orders delivery_days. This optional field may not be known when an order is first recorded. It also inserts 35 duplicate business-event groups under different order IDs. The customer, date, product, quantity and unit price match, simulating a transaction imported more than once. Since the surrogate IDs remain unique, the audit groups by business attributes rather than relying only on primary-key duplication. The fixed random seed makes the cases reproducible.

8. SQL analysis and business value

The accompanying SQL file demonstrates joins, filters, grouping, aggregate functions and sorting. Queries calculate completed-sale revenue by brand and month, identify popular models, compare sales across age bands and payment methods, rank store performance and flag stock at or below its reorder level. Separate audit queries count missing delivery estimates and repeated business events; PRAGMA foreign_key_check validates referential integrity.

9. Ethics, privacy and responsible use

All records are fictional. Email addresses use the reserved example.org domain. No real telephone numbers, payment-card details or IMEI identifiers are stored. The results must not be presented as evidence about actual customers or the UK phone market. A retailer using real customer information would need to comply with the UK GDPR and Data Protection Act 2018: explain why data is collected, collect only what is necessary, maintain accuracy, protect access, set retention periods and support data-subject rights. Marketing preferences should be separate from information needed to fulfil an order; buying a phone should not automatically count as marketing consent. Payment-card details should be handled by a suitable payment provider rather than stored in this database.

10. Limitations and conclusion

This is a generated practice dataset and cannot establish real market share or customer preferences. A live system would also need access controls, audit logging, backups and further entities such as suppliers, returns and stock movements. The six-table design demonstrates relational links, two composite primary keys, meaningful constraints, all four measurement scales and reproducible data-quality checks. It therefore provides a practical foundation for retail transaction and inventory analysis.